

4. System Analysis & Design

4.1 Problem Statement & Objectives

4.1.1 Problem Statement

The pharmaceutical market experiences frequent price fluctuations due to inflation, currency exchange rates, manufacturing costs, import regulations, and supply chain disruptions. As a result, patients, pharmacists, and healthcare professionals often struggle to access up-to-date medicine prices and anticipate future price changes.

Traditional pharmacy management systems primarily provide static information regarding available medicines and their current prices. They generally lack intelligent capabilities such as future price forecasting, historical price trend analysis, and AI-assisted medical guidance. Furthermore, medicine information is often scattered across multiple sources, requiring users to manually search for relevant information.

To address these limitations, this project proposes an intelligent Medicine Price Prediction System that combines historical drug pricing data, machine learning forecasting, and Retrieval-Augmented Generation (RAG) powered by a Large Language Model (LLM). The system enables users to search medicines, visualize historical price trends, predict future prices, and obtain contextual pharmaceutical guidance through an AI assistant grounded in a custom medical knowledge base.

The proposed solution integrates several technologies into a unified platform, including FastAPI for backend services, SQLite for local data storage, Prophet for time-series forecasting, and the Qwen 2.5 3B-Instruct language model enhanced with FAISS-based semantic retrieval. The user interface is implemented using Streamlit, providing an interactive dashboard for searching medicines, viewing analytical charts, generating forecasts, and communicating with the AI assistant.

4.1.2 Project Objectives

The primary objective of this project is to design and implement an intelligent pharmaceutical information system capable of assisting users in accessing medicine information, analyzing historical prices, forecasting future prices, and providing AI-powered pharmaceutical recommendations.

The specific objectives of the project are:

General Objectives

- Develop an integrated medicine information management system.
- Provide an intuitive interface for searching and exploring pharmaceutical data.
- Improve accessibility to medicine pricing information.
- Support data-driven decision making using machine learning techniques.
- Integrate artificial intelligence to improve user interaction and information retrieval.

Functional Objectives

The system is designed to achieve the following functional objectives:

- Retrieve medicine information from an external drug information API.
- Cache retrieved medicine data locally in SQLite to improve performance and reduce unnecessary API requests.
- Allow users to search medicines using multiple filtering criteria.
- Display detailed medicine information including:
 - Commercial name
 - Scientific name
 - Manufacturer
 - Drug class
 - Administration route
 - Current price
- Store historical medicine prices for future analysis.
- Visualize historical price changes using interactive charts.
- Predict future medicine prices using Prophet forecasting models.
- Display prediction confidence intervals.
- Inform users whether a forecasting model exists for a selected medicine.
- Provide an AI-powered chatbot capable of answering medicine and symptom-related questions.
- Restrict chatbot responses to information contained within the project's custom pharmaceutical knowledge base using Retrieval-Augmented Generation (RAG).

Machine Learning Objectives

The machine learning module aims to:

- Preprocess historical medicine pricing datasets.
- Generate clean time-series data suitable for forecasting.
- Train individual Prophet forecasting models for medicines with sufficient historical records.
- Store trained forecasting models for future inference.
- Generate future price forecasts for user-selected forecasting periods.
- Produce upper and lower confidence intervals for prediction reliability.

Artificial Intelligence Objectives

The AI assistant aims to:

- Deploy the Qwen 2.5 3B-Instruct Large Language Model.
- Implement Retrieval-Augmented Generation (RAG) to minimize hallucination.
- Build a searchable pharmaceutical knowledge base.
- Convert knowledge documents into semantic vector embeddings.
- Store embeddings using FAISS vector indexing.
- Retrieve the most relevant document chunks before generating responses.
- Ensure chatbot responses remain grounded in verified pharmaceutical information instead of relying solely on the language model's internal knowledge.

User Experience Objectives

The Streamlit application is designed to:

- Provide a clean and intuitive dashboard.
- Allow users to search medicines quickly.
- Display interactive price history visualizations.
- Present forecasting results using dynamic charts.
- Offer a conversational AI interface for pharmaceutical inquiries.
- Maintain responsive interaction with backend services.

Technical Objectives

The project also aims to:

- Implement a modular layered software architecture.
- Separate presentation, business logic, data access, and machine learning components.
- Develop RESTful APIs using FastAPI.
- Utilize SQLite for lightweight local data persistence.
- Enable communication between independent services through HTTP APIs.
- Provide scalable architecture that can be migrated to larger database systems if required.
- Facilitate future deployment of cloud-hosted AI services.

4.1.3 Project Scope

The scope of the Medicine Price Prediction System includes the following functionalities:

Included Scope

- Medicine search and filtering.
- Medicine detail retrieval.
- Historical price visualization.
- Price forecasting using Prophet.
- AI-powered pharmaceutical assistant.
- Retrieval-Augmented Generation using a custom knowledge base.
- Local SQLite database management.
- RESTful backend APIs.
- Streamlit web dashboard.
- External pharmaceutical API integration.

Excluded Scope

The following features are outside the scope of the current implementation:

- Online medicine purchasing.
- Electronic payment processing.
- User authentication and authorization.
- Patient medical record management.
- Doctor appointment scheduling.
- Prescription generation.
- Inventory management for pharmacies.
- Real-time synchronization with national healthcare systems.
- Automatic online model retraining after deployment.

4.1.4 Expected Outcomes

Upon successful completion of the project, the system is expected to:

- Reduce the effort required to locate medicine information.
- Provide users with historical price insights.
- Forecast future medicine prices with acceptable accuracy using time-series analysis.
- Improve pharmaceutical information accessibility through conversational AI.
- Demonstrate the practical integration of Machine Learning, Large Language Models, Retrieval-Augmented Generation, and RESTful web services into a unified healthcare application.
- Provide a scalable foundation for future intelligent pharmaceutical management systems.

4.2 Use Case Diagram & Descriptions

4.2.1 Overview

The Use Case Diagram illustrates the functional interactions between the system and its external actors. It identifies the services provided by the Medicine Price Prediction System and demonstrates how users interact with different modules including medicine search, price prediction, and the AI assistant.

The system communicates with external services, including a Drug Information API for retrieving medicine data and a separate AI service hosting the Qwen 2.5 language model with Retrieval-Augmented Generation (RAG). Users interact with the system exclusively through the Streamlit web interface, while backend services coordinate data retrieval, prediction, and database operations.

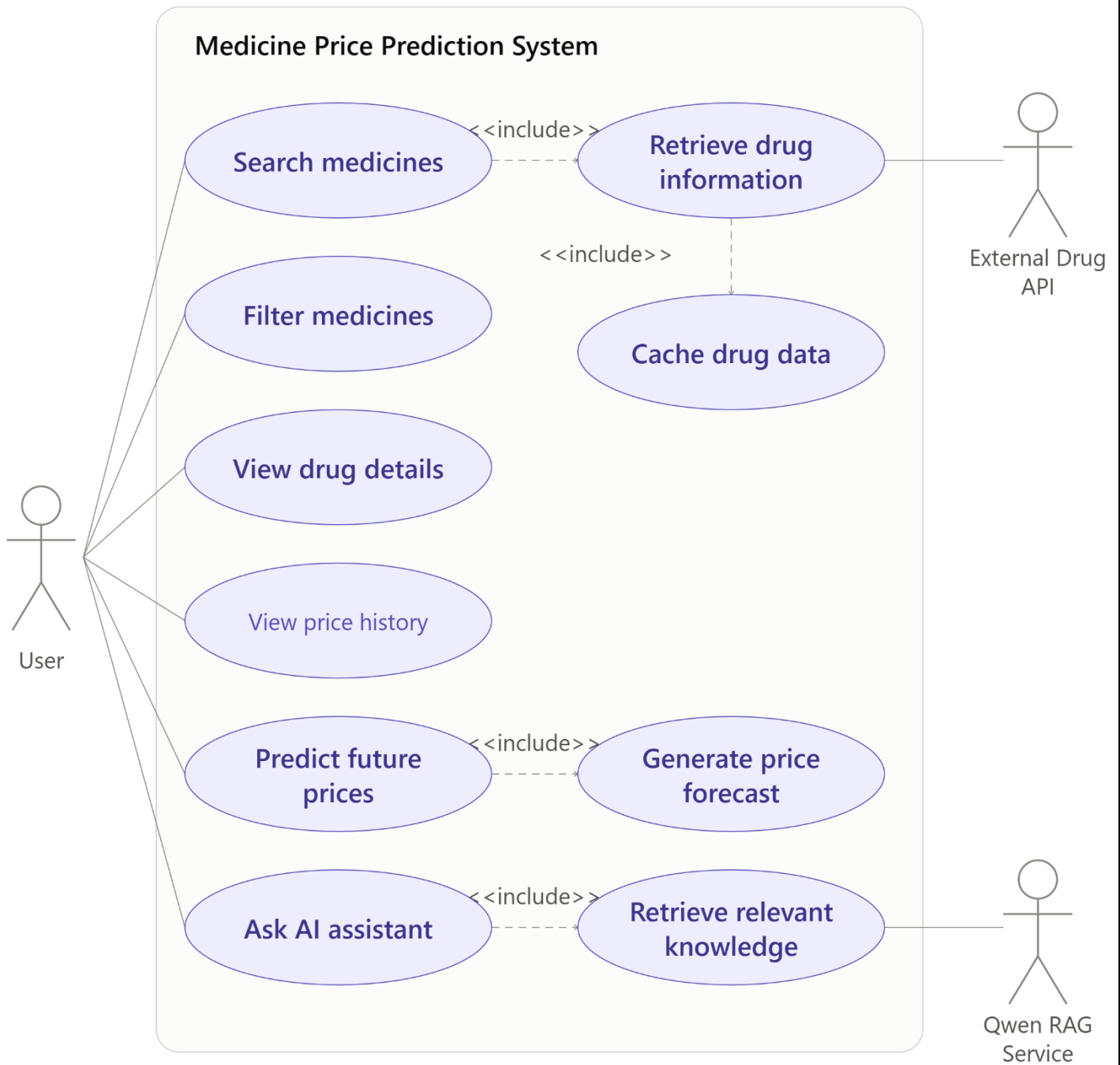
4.2.2 System Actors

Actor	Description
User	Any person using the application to search medicines, view drug information, predict future prices, or interact with the AI assistant.
External Drug API	Third-party API supplying medicine information that is cached locally in SQLite.
Qwen RAG Service	Independent AI service responsible for answering pharmaceutical questions using Retrieval-Augmented Generation.

4.2.3 List of Use Cases

ID	Use Case	Primary Actor
UC-01	Search Medicines	User
UC-02	Filter Medicines	User
UC-03	View Drug Details	User
UC-04	View Price History	User
UC-05	Predict Future Prices	User
UC-06	Ask AI Assistant	User
UC-07	Retrieve Drug Information	External Drug API
UC-08	Cache Drug Data	System
UC-09	Generate Price Forecast	Prophet Prediction Engine
UC-10	Retrieve Knowledge using RAG	Qwen AI Service

4.2.4 Use Case Diagram



4.2.5 Use Case Descriptions

UC-01 Search Medicines

Item	Description
Use Case ID	UC-01
Use Case Name	Search Medicines
Primary Actor	User
Goal	Search for medicines stored in the database using keywords or filters.
Preconditions	System is running and database is available.
Trigger	User enters a search term and clicks Search.
Main Flow	1. User enters search keyword. 2. Frontend sends request to FastAPI. 3. Backend queries SQLite. 4. If data is unavailable, backend requests the External Drug API. 5. Results are cached. 6. Matching medicines are displayed.
Alternative Flow	No matching medicines are found.
Postconditions	Matching medicines are displayed to the user.

UC-02 Filter Medicines

Item	Description
Use Case ID	UC-02
Use Case Name	Filter Medicines
Primary Actor	User
Goal	Narrow search results using available filters.
Preconditions	Medicine list exists.
Trigger	User selects one or more filters.
Main Flow	1. User selects manufacturer or route. 2. Backend applies filters. 3. Filtered results are displayed.
Postconditions	Filtered medicine list is shown.

UC-03 View Drug Details

Item	Description
Use Case ID	UC-03
Use Case Name	View Drug Details
Primary Actor	User
Goal	Display detailed medicine information.
Preconditions	Medicine exists.
Trigger	User clicks Details.
Main Flow	1. Frontend requests medicine details. 2. Backend retrieves medicine data. 3. Historical prices are loaded. 4. Complete information is displayed.
Postconditions	Drug information becomes available for review.

UC-04 View Price History

Item	Description
Use Case ID	UC-04
Use Case Name	View Price History
Primary Actor	User
Goal	Visualize historical medicine prices.
Preconditions	Historical data exists.
Trigger	Drug details page is opened.
Main Flow	1. Backend retrieves historical records. 2. Streamlit generates a Plotly chart. 3. User explores price evolution over time.
Alternative Flow	No historical data available.
Postconditions	Historical chart is displayed.

UC-05 Predict Future Prices

Item	Description
Use Case ID	UC-05
Use Case Name	Predict Future Prices
Primary Actor	User
Goal	Forecast future medicine prices.
Preconditions	Forecasting model has been trained.
Trigger	User clicks Predict.
Main Flow	1. User selects forecast duration. 2. Backend loads Prophet model. 3. Prophet generates predictions. 4. Confidence intervals are calculated. 5. Forecast chart is displayed.
Alternative Flow	No trained model exists.
Postconditions	Future prices are visualized.

UC-06 Ask AI Assistant

Item	Description
Use Case ID	UC-06
Use Case Name	Ask AI Assistant
Primary Actor	User
Goal	Obtain medicine or symptom-related information.
Preconditions	AI service is available.
Trigger	User submits a question.
Main Flow	1. User sends prompt. 2. Streamlit forwards request to Qwen API. 3. RAG retrieves relevant document chunks from FAISS. 4. Retrieved context is added to the prompt. 5. Qwen generates an answer. 6. Response is displayed.
Alternative Flow	AI service unavailable.
Postconditions	User receives grounded AI response.

UC-07 Retrieve Drug Information

Item	Description
Use Case ID	UC-07
Use Case Name	Retrieve Drug Information
Primary Actor	External Drug API
Goal	Supply medicine information requested by the backend.
Preconditions	Internet connection available.
Trigger	Backend cache miss occurs.
Main Flow	Backend requests medicine information. API returns structured data. Data is validated.
Postconditions	Backend receives latest medicine information.

UC-08 Cache Drug Data

Item	Description
Use Case ID	UC-08
Use Case Name	Cache Drug Data
Primary Actor	System
Goal	Store retrieved medicine information locally.
Preconditions	Valid API response received.
Trigger	External API response returned.
Main Flow	Backend validates response. Drug data is stored in SQLite. Future searches use local cache.
Postconditions	Cached records become available.

UC-09 Generate Price Forecast

Item	Description
Use Case ID	UC-09
Use Case Name	Generate Price Forecast
Primary Actor	Prophet Prediction Engine
Goal	Produce future price estimates.
Preconditions	Historical data exists.
Trigger	Prediction request received.
Main Flow	Historical data loaded. Prophet performs forecasting. Predictions returned to backend.
Postconditions	Forecast generated successfully.

UC-10 Retrieve Relevant Knowledge

Item	Description
Use Case ID	UC-10
Use Case Name	Retrieve Relevant Knowledge
Primary Actor	Qwen RAG Service
Goal	Retrieve document chunks relevant to the user's question.
Preconditions	FAISS index available.
Trigger	Chat request received.
Main Flow	User query embedded, Similarity search performed. Top-3 document chunks retrieved. Context passed to Qwen, Response generated.
Postconditions	AI response grounded in project knowledge base.

4.2.6 Use Case Relationships

- **Search Medicines** includes **Retrieve Drug Information** when requested data is unavailable in the local cache.
- **Retrieve Drug Information** includes **Cache Drug Data** to improve future search performance.
- **Predict Future Prices** includes **Generate Price Forecast**, which invokes the Prophet forecasting model.
- **Ask AI Assistant** includes **Retrieve Relevant Knowledge**, ensuring that the Qwen model generates responses based on retrieved pharmaceutical information rather than relying solely on its pretrained knowledge.

4.3 Functional & Non-Functional Requirements

This section defines the functional capabilities that the Medicine Price Prediction System must provide, as well as the quality attributes and operational constraints that ensure reliable, efficient, and secure system performance.

4.3.1 Functional Requirements

Functional requirements describe the services and operations that the system performs to satisfy user needs.

FR-01 Medicine Search

Description

The system shall allow users to search medicines using a keyword.

Inputs

- Medicine name
- Scientific name (if applicable)

Processing

- Validate user input.
- Search the local SQLite database.
- If data is unavailable, retrieve it from the External Drug API.
- Cache newly retrieved data locally.

Outputs

- List of matching medicines.

FR-02 Medicine Filtering

Description

The system shall allow users to filter medicines based on available attributes.

Supported filters include:

- Manufacturer
- Route of administration
- Search keyword
- Pagination

Outputs

Filtered medicine list.

FR-03 Display Drug Information

Description

The system shall display complete information for a selected medicine.

Displayed information includes:

- Commercial name
- Scientific name
- Manufacturer
- Drug class
- Administration route
- Current price
- Historical prices
- Model availability status

FR-04 Display Historical Price Chart

Description

The system shall visualize historical medicine prices using interactive charts.

Features include:

- Time-series visualization
- Zooming
- Hover tooltips
- Automatic date formatting

Visualization Library:

- Plotly

FR-05 Future Price Prediction

Description

The system shall forecast future medicine prices using Prophet.

Inputs:

- Drug ID
- Forecast period (7–90 days)

Outputs:

- Predicted prices
- Prediction dates
- Upper confidence interval
- Lower confidence interval

FR-06 Prediction Availability Detection

The system shall determine whether a trained Prophet model exists before allowing prediction.

If unavailable:

- Display warning message.
- Disable forecasting functionality.

FR-07 AI Chat Assistant

The system shall provide an AI-powered pharmaceutical assistant capable of answering user questions.

Supported topics include:

- Symptoms
- Medicines
- Drug usage
- Pharmaceutical information

FR-08 Retrieval-Augmented Generation (RAG)

The chatbot shall retrieve relevant information from the custom pharmaceutical knowledge base before generating responses.

The retrieval pipeline shall:

- Embed the user's question.
- Search the FAISS vector database.
- Retrieve the three most relevant document chunks.
- Append retrieved context to the LLM prompt.

FR-09 Knowledge-Based Responses

The chatbot shall generate responses only using retrieved knowledge whenever relevant information exists.

If information does not exist inside the knowledge base, the chatbot shall clearly inform the user that the requested information is unavailable instead of generating unsupported answers.

FR-10 External Drug API Integration

The backend shall communicate with an external pharmaceutical API to retrieve medicine information.

The backend shall:

- Validate API responses.
- Transform received data.
- Store valid information locally.

FR-11 Local Data Caching

The system shall cache medicine data retrieved from the external API.

Objectives:

- Reduce API calls.
- Improve response time.
- Enable offline querying for previously retrieved medicines.

FR-12 Database Management

The backend shall maintain local SQLite tables for:

- Drug information
- Historical prices
- Cached API responses

FR-13 REST API Services

The backend shall expose RESTful API endpoints for:

- Medicine search
- Medicine details
- Forecast generation

These APIs are consumed by the Streamlit frontend.

FR-14 Interactive Dashboard

The frontend shall provide an interactive dashboard allowing users to:

- Search medicines
- View detailed information
- Generate forecasts
- Explore historical charts
- Chat with the AI assistant

FR-15 Error Handling

The system shall display meaningful messages when:

- Drug not found
- Prediction unavailable
- AI service offline
- Backend unavailable
- External API unreachable

FR-16 Service Health Monitoring

The frontend shall periodically verify the availability of:

- FastAPI Backend
- Qwen AI Service

Status indicators shall be displayed in the sidebar.

FR-17 Session Management

The frontend shall preserve:

- Selected medicine
- Prediction state
- Chat history

using Streamlit Session State during the active user session.

4.3.2 Functional Requirements Summary

ID	Requirement
FR-01	Search medicines
FR-02	Filter medicines
FR-03	Display medicine details
FR-04	Visualize historical prices
FR-05	Predict future prices
FR-06	Detect forecasting model availability
FR-07	AI chatbot
FR-08	Retrieval-Augmented Generation
FR-09	Knowledge-grounded responses
FR-10	External Drug API integration
FR-11	Local caching
FR-12	SQLite database management
FR-13	REST API services
FR-14	Interactive dashboard
FR-15	Error handling
FR-16	Service health monitoring
FR-17	Session management

4.3.3 Non-Functional Requirements

Non-functional requirements define the quality attributes that determine how well the system performs its functions.

Performance

The system shall provide fast response times under normal operating conditions.

Requirements:

- Medicine search ≤ 2 seconds (cached data).
- Drug details retrieval ≤ 2 seconds.
- Historical chart generation ≤ 3 seconds.
- AI response generation depends on the Qwen model but should typically complete within 30 seconds.
- Forecast generation should complete within a few seconds for a single medicine.

Scalability

The system architecture shall support future expansion by allowing:

- Migration from SQLite to PostgreSQL or MySQL.
- Addition of new machine learning models.
- Expansion of the pharmaceutical knowledge base.
- Deployment of cloud-hosted AI services.

Reliability

The system shall remain operational despite temporary failures of external services.

Specifically:

- Cached medicine data shall remain accessible if the External Drug API is unavailable.
- The frontend shall notify users if the AI service is offline.
- Prediction services shall continue operating independently of the chatbot service.

Availability

The backend services shall be available whenever the application is running.

Independent services include:

- FastAPI Backend
- Qwen AI Service

Each service shall operate independently to reduce system-wide failures.

Maintainability

The software shall follow a modular layered architecture to simplify maintenance and future enhancements.

Project modules include:

- Routers
- Services
- Repositories
- Models
- Schemas
- Core configuration
- Machine Learning module
- Streamlit frontend

This separation reduces coupling and improves code readability.

Usability

The user interface shall be simple and intuitive.

Design objectives include:

- Minimal learning curve.
- Clear navigation.
- Interactive charts.
- Readable medicine information.
- Simple AI conversation interface.
- Immediate visual feedback.

Security

The current implementation does not require user authentication.

However, the system shall:

- Validate incoming requests.
- Handle invalid inputs gracefully.
- Prevent server crashes caused by malformed data.
- Restrict Cross-Origin Resource Sharing (CORS) according to backend configuration.
- Protect internal APIs from unintended misuse.

Portability

The application shall be deployable on:

- Windows
- Linux
- macOS

The use of Python, FastAPI, SQLite, and Streamlit ensures cross-platform compatibility.

Compatibility

The system shall support modern web browsers through the Streamlit interface, including:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox

Extensibility

The architecture shall allow future integration of:

- Additional forecasting models.
- Larger language models.
- Expanded pharmaceutical databases.
- User authentication.
- Pharmacy inventory management.
- Online medicine purchasing.
- Mobile applications.

Accuracy

Forecasting accuracy depends on:

- Historical price quality.
- Availability of sufficient time-series observations.
- Prophet forecasting performance.

AI response accuracy depends on:

- Quality of the knowledge base.
- Semantic retrieval effectiveness.
- Relevance of retrieved document chunks.

The Retrieval-Augmented Generation pipeline significantly reduces hallucinations by grounding responses in verified pharmaceutical documents.

Data Integrity

The system shall preserve data consistency by:

- Validating incoming API responses.
- Storing historical prices without duplication.
- Maintaining referential integrity between medicine records and their associated price history.

Maintainability Metrics

The project adopts several software engineering practices to improve maintainability:

- Layered architecture.
- Modular code organization.
- Separation of concerns.
- Reusable service classes.
- Repository pattern for database access.
- Centralized configuration management.

4.3.4 Non-Functional Requirements Summary

Category	Requirement
Performance	Fast response for searches, predictions, and chart rendering
Reliability	Independent services with graceful failure handling
Availability	Backend and AI services operate independently
Maintainability	Modular layered architecture
Scalability	Supports migration to larger databases and cloud deployment
Security	Input validation, CORS configuration, error handling
Usability	Interactive and user-friendly Streamlit interface
Compatibility	Runs on modern web browsers and operating systems
Portability	Cross-platform deployment
Extensibility	Supports future AI, ML, and feature expansion
Accuracy	Prophet forecasting and RAG-based grounded AI responses
Data Integrity	Consistent storage and validated database operations

4.4 Software Architecture

Architecture Style

The Medicine Price Prediction System follows a **modular layered architecture**. The application separates presentation, business logic, data access, machine learning, and artificial intelligence into independent modules, making the system easier to maintain, extend, and test.

The architecture consists of four major layers:

Presentation Layer

Implemented using **Streamlit**, this layer provides the graphical user interface where users can:

- Search medicines
- View medicine information
- Display historical prices
- Generate future price forecasts
- Interact with the AI pharmaceutical assistant

The frontend communicates only with REST APIs and does not access the database directly.

Backend Layer

Implemented using **FastAPI**.

Responsibilities include:

- Handling HTTP requests
- Business logic
- Data validation
- Medicine search
- Database operations
- Prediction requests
- Communication with the external medicine API

Backend modules include:

- Routers
- Services
- Repositories
- Schemas
- Models

Machine Learning Layer

The ML subsystem predicts future medicine prices.

Main components:

- preprocessing.py
- trainer.py
- predictor.py
- model_manager.py
- train_models.py

The forecasting model uses **Meta Prophet**, which analyzes historical medicine prices and predicts future prices together with confidence intervals.

Prediction requests are initiated by the backend and returned as JSON responses.

AI Assistant Layer

The chatbot is completely separated from the backend.

It is implemented using:

- Qwen 2.5 3B Instruct
- Retrieval-Augmented Generation (RAG)
- FAISS Vector Database
- Sentence Transformers

The chatbot answers pharmaceutical questions using a custom medical knowledge base instead of relying solely on the language model.

Data Layer

The system uses SQLite for persistent storage.

Main stored data includes:

- Drug information
- Cached API responses
- Historical prices
- Prediction metadata

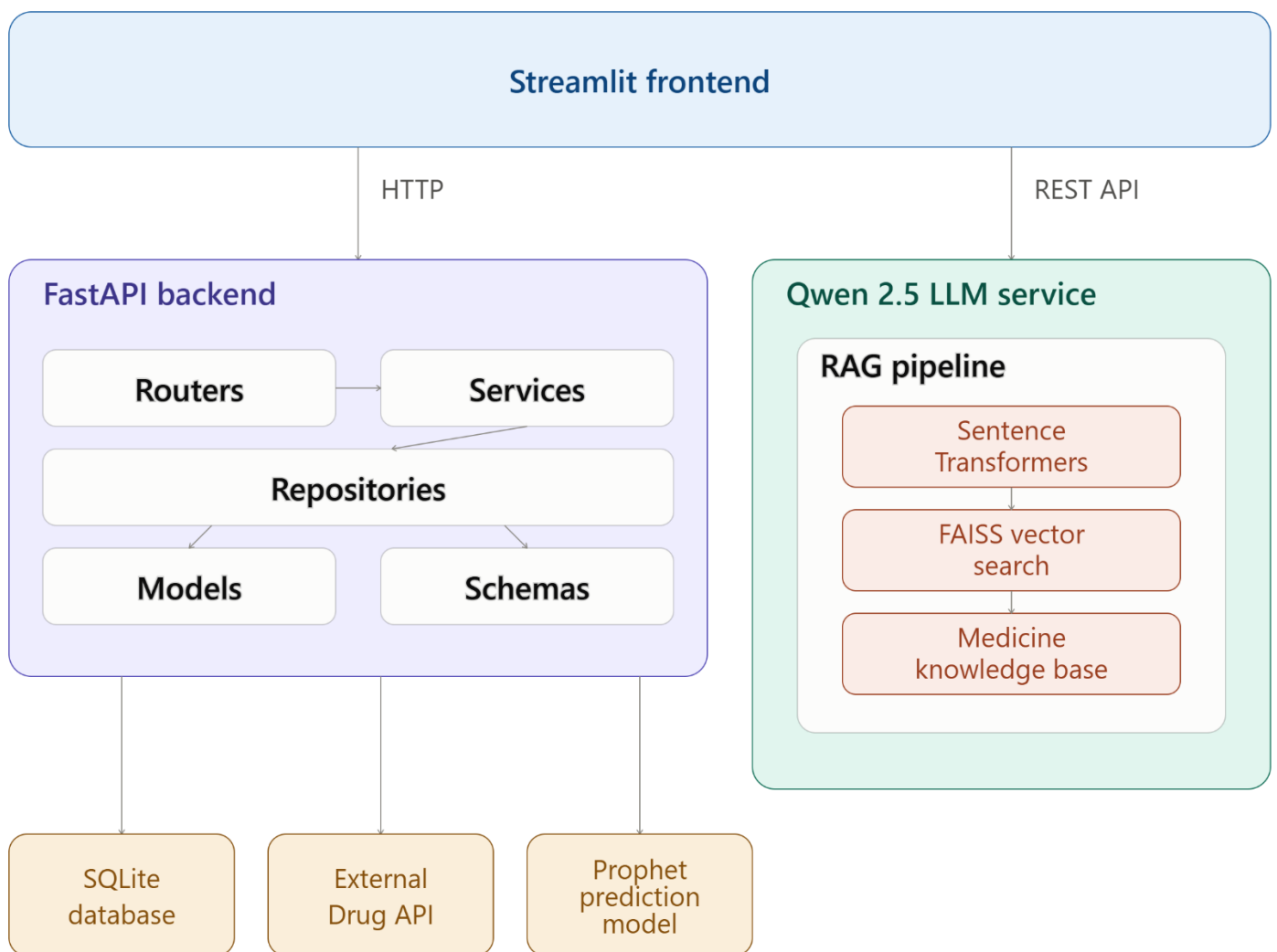
SQLite was selected because it is lightweight, serverless, portable, and sufficient for the project size.

External Services

The system communicates with:

- External Drug API
- HuggingFace model files (during model setup)
- Cloudflare Tunnel (public chatbot endpoint)

Architecture Diagram



4.5 Database Design & Data Modeling

Database Overview

The project stores application data in SQLite.

The database is organized into normalized tables that separate medicine information from historical price records.

This design minimizes redundancy while maintaining efficient querying performance.

Main Tables

drugs

Stores the latest information for every medicine.

Main attributes include:

- id
- commercial_name_en
- scientific_name
- manufacturer
- route
- drug_class
- price_egp

price_history

Stores historical medicine prices.

Attributes include:

- id
- drug_id
- recorded_at
- price_egp

Each medicine can have multiple historical prices.

raw_api_response

Stores cached responses received from the external drug API.

Purpose:

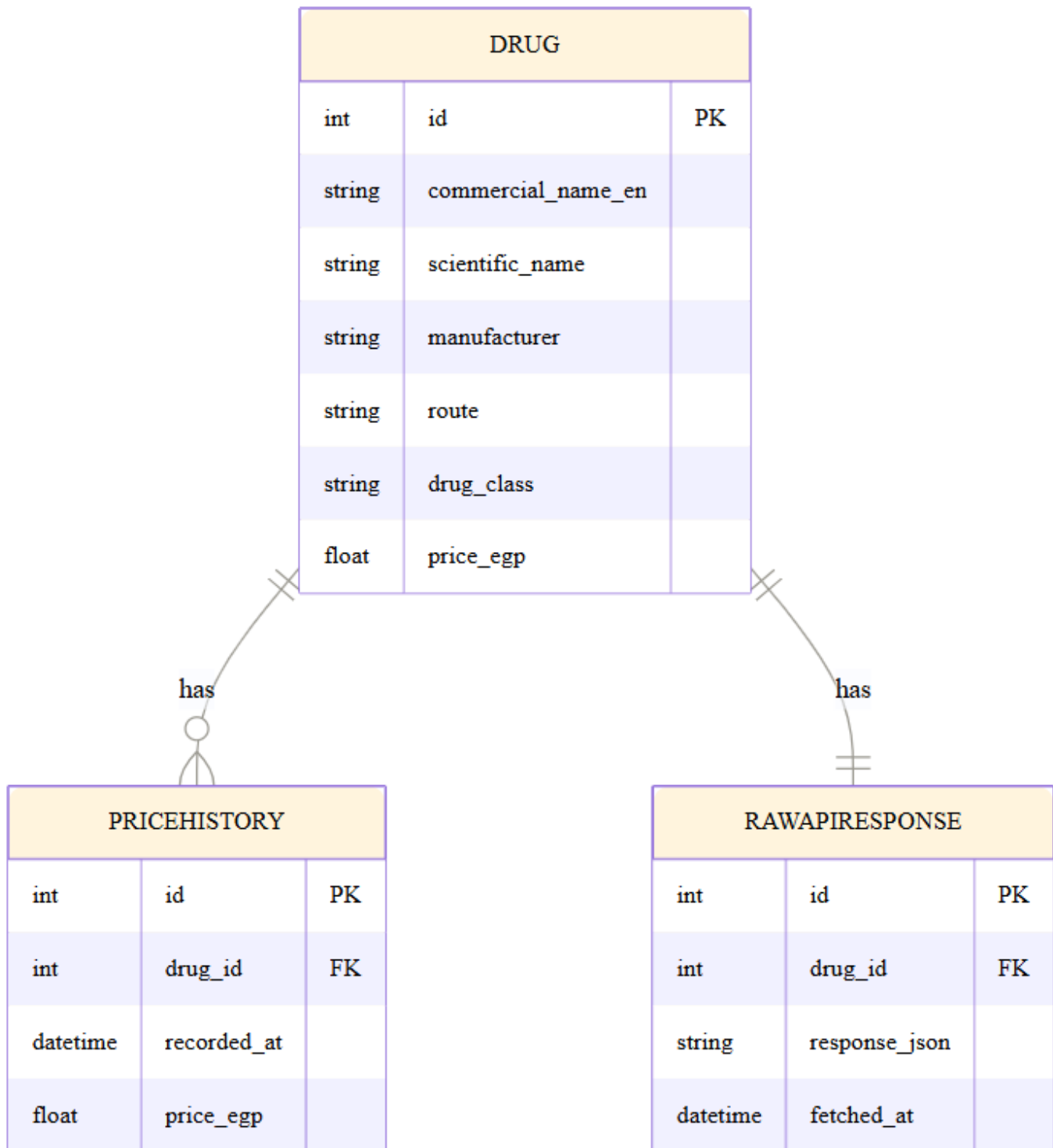
- Reduce repeated API calls
- Improve response time
- Preserve original external data

Relationships

One Drug → Many Price History Records

Drug → One Cached API Response

ER Diagram



4.6 Logical & Physical Schema

Primary Keys

- Drug.id
- PriceHistory.id
- RawApiResponse.id

Each table uses an auto-increment integer primary key to uniquely identify records.

Foreign Keys

PriceHistory.drug_id

references

Drug.id

RawApiResponse.drug_id

references

Drug.id

These relationships ensure referential integrity between medicine records and their associated historical data.

Indexes

Indexes are created on frequently queried columns to improve search performance:

- commercial_name_en
- scientific_name
- manufacturer
- route
- drug_id (Price History)
- recorded_at

Composite Unique Constraints

To prevent duplicate historical entries, a composite unique constraint is applied on:
(drug_id, recorded_at)

This ensures that only one price record exists for a specific medicine at a specific timestamp.

Third Normal Form (3NF)

The database follows Third Normal Form:

- Each table represents a single entity.
- Non-key attributes depend only on the primary key.
- No transitive dependencies exist.
- Historical prices are separated from medicine metadata to eliminate redundancy.

Optimized Storage for Time-Series Price History

Historical prices are stored in a dedicated table rather than within the drug table.

Advantages include:

- Efficient insertion of new price records.
- Reduced duplication of medicine information.
- Faster historical trend analysis.
- Better scalability for forecasting models such as Prophet.
- Simplified retrieval of chronological price sequences.

4.7 Data Flow & System Behavior

Context-Level DFD

External entities:

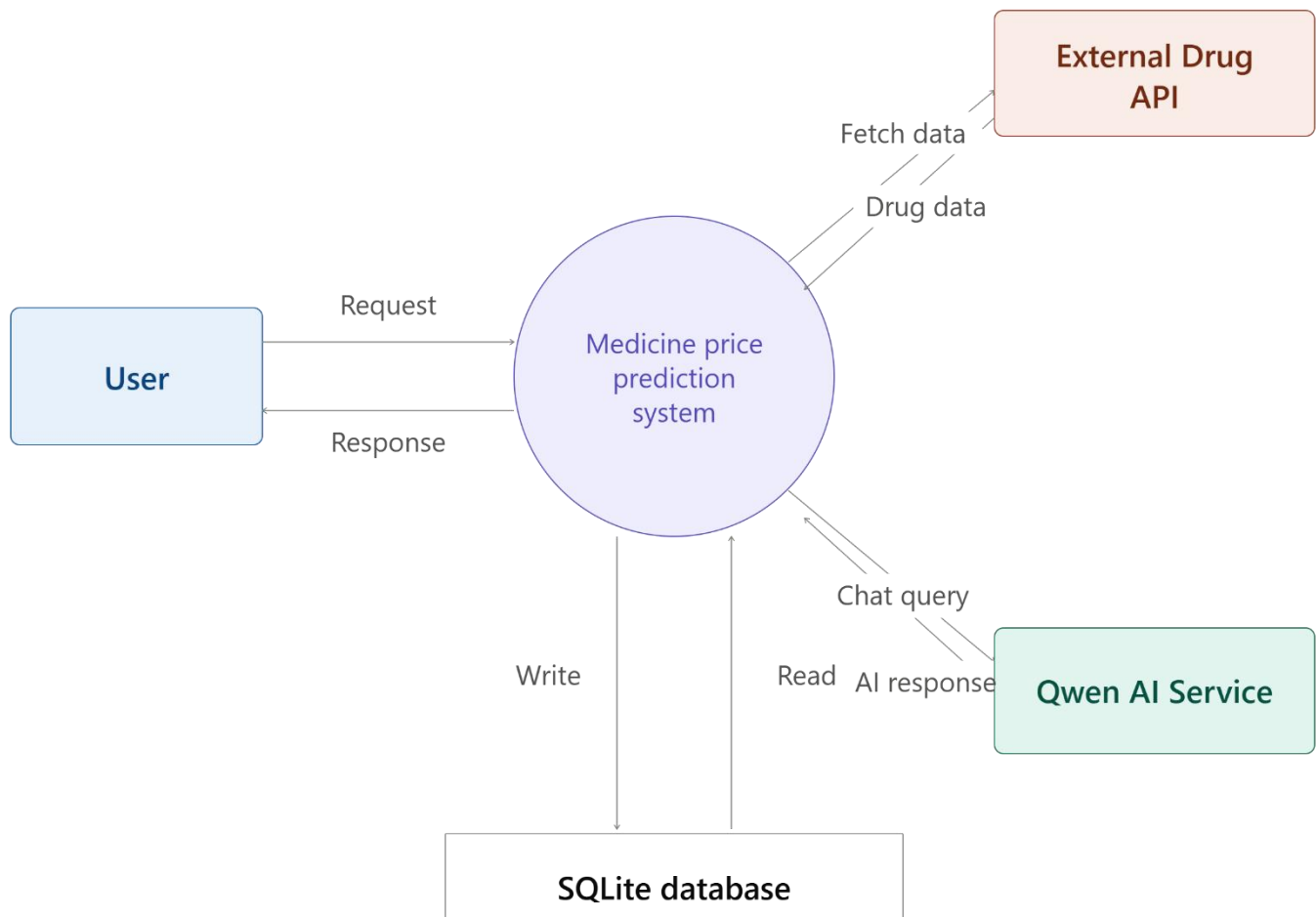
- User
- External Drug API
- Qwen AI Service

Processes:

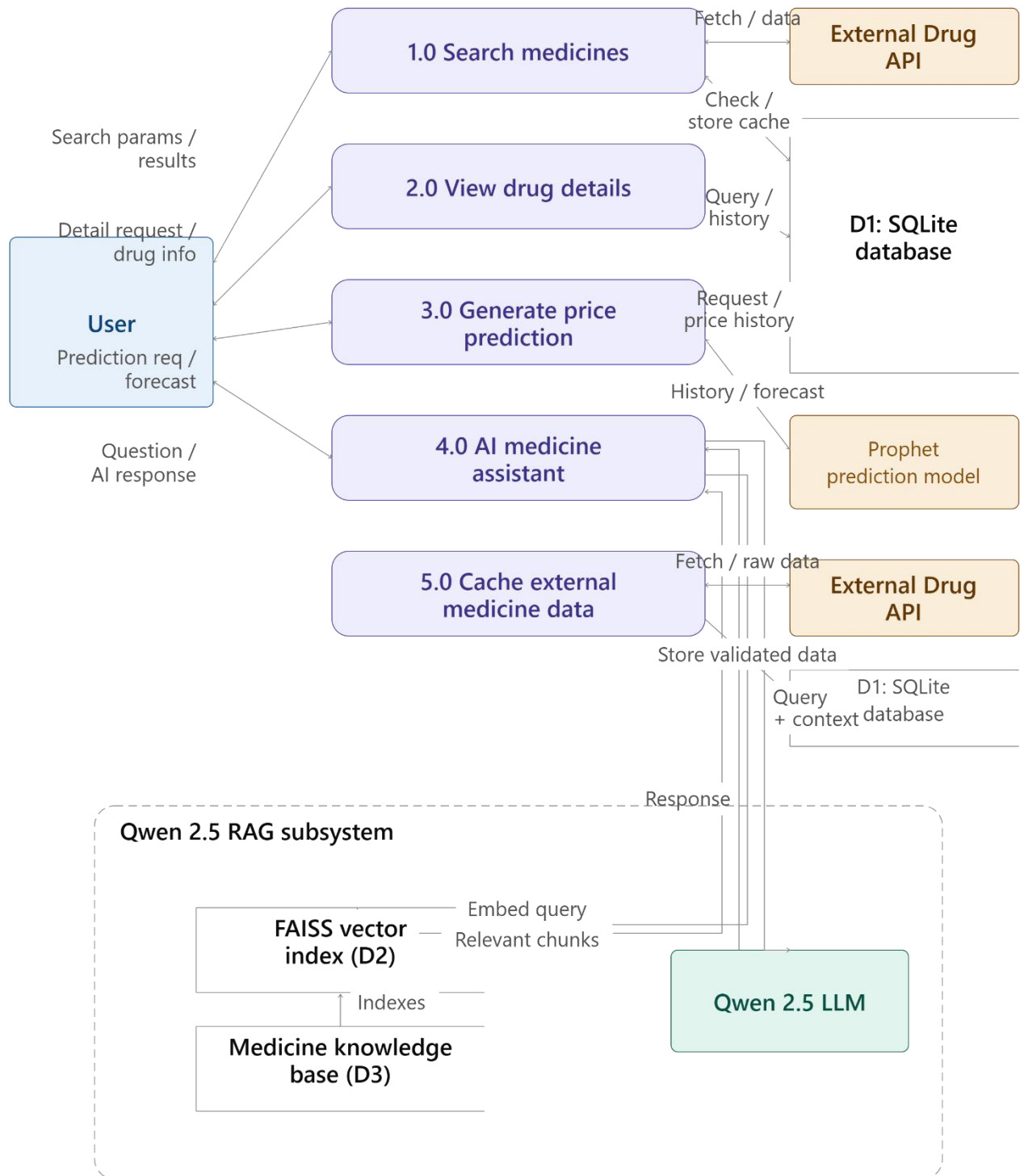
- Medicine Search
- Price Prediction
- AI Chat

Data Store:

- SQLite Database

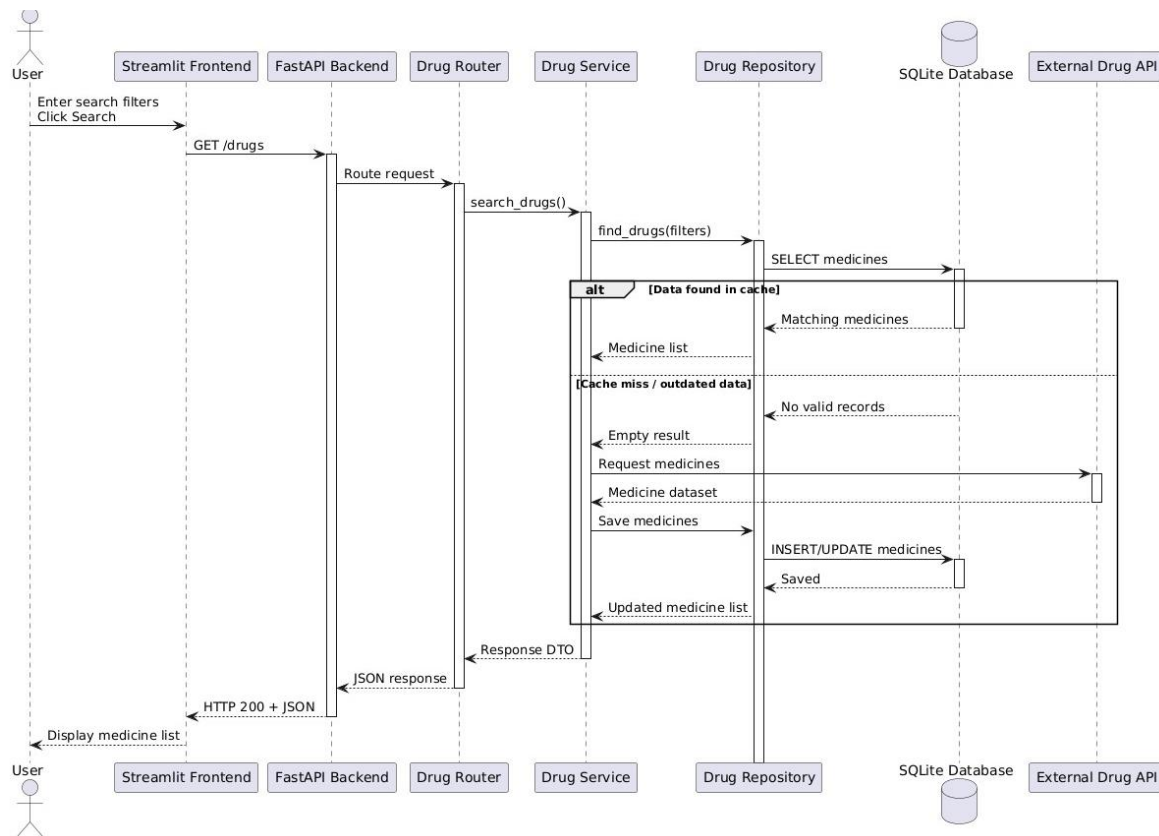


Level 1 DFD

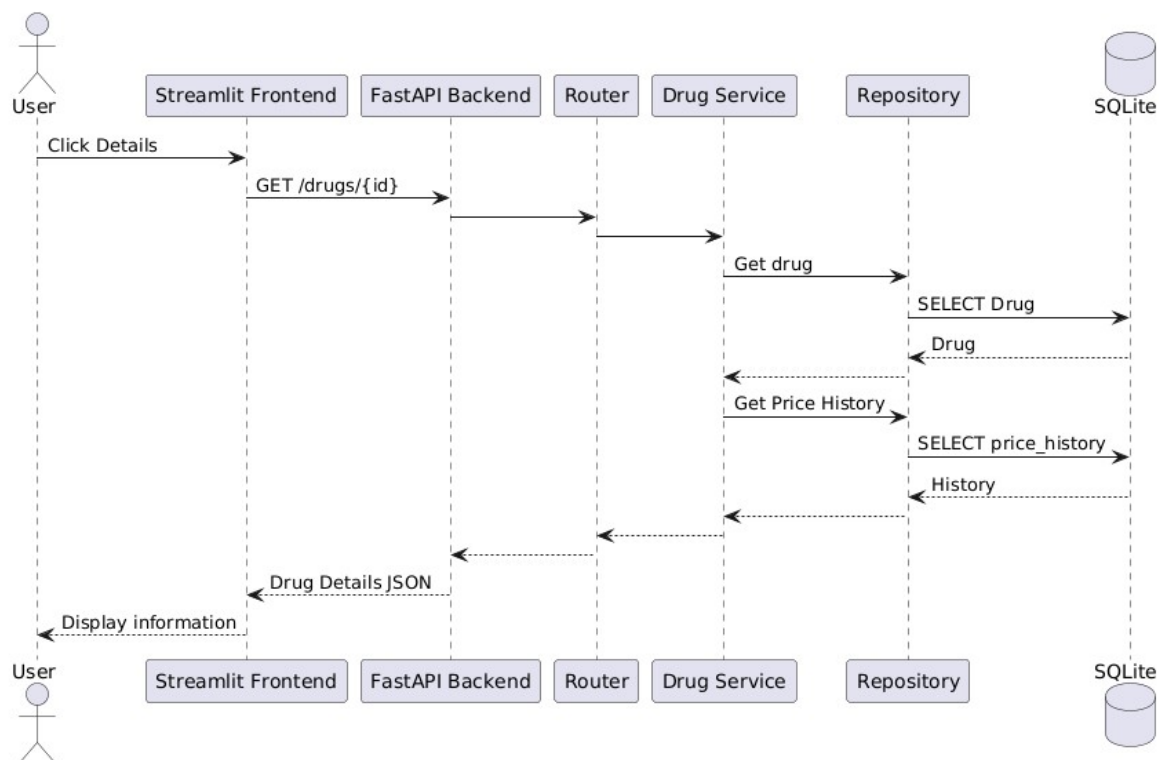


4.8 Sequence Diagrams

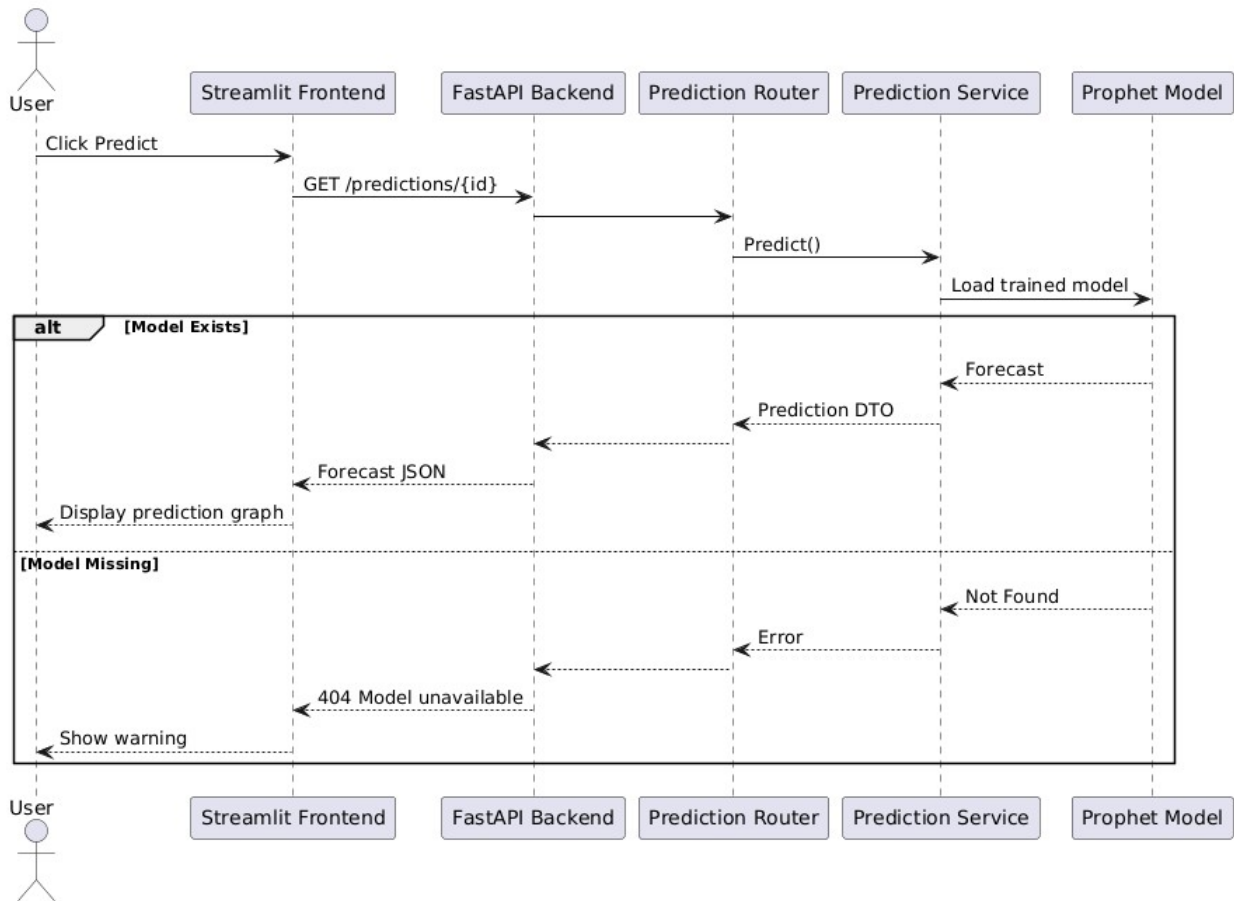
A. Drug Search



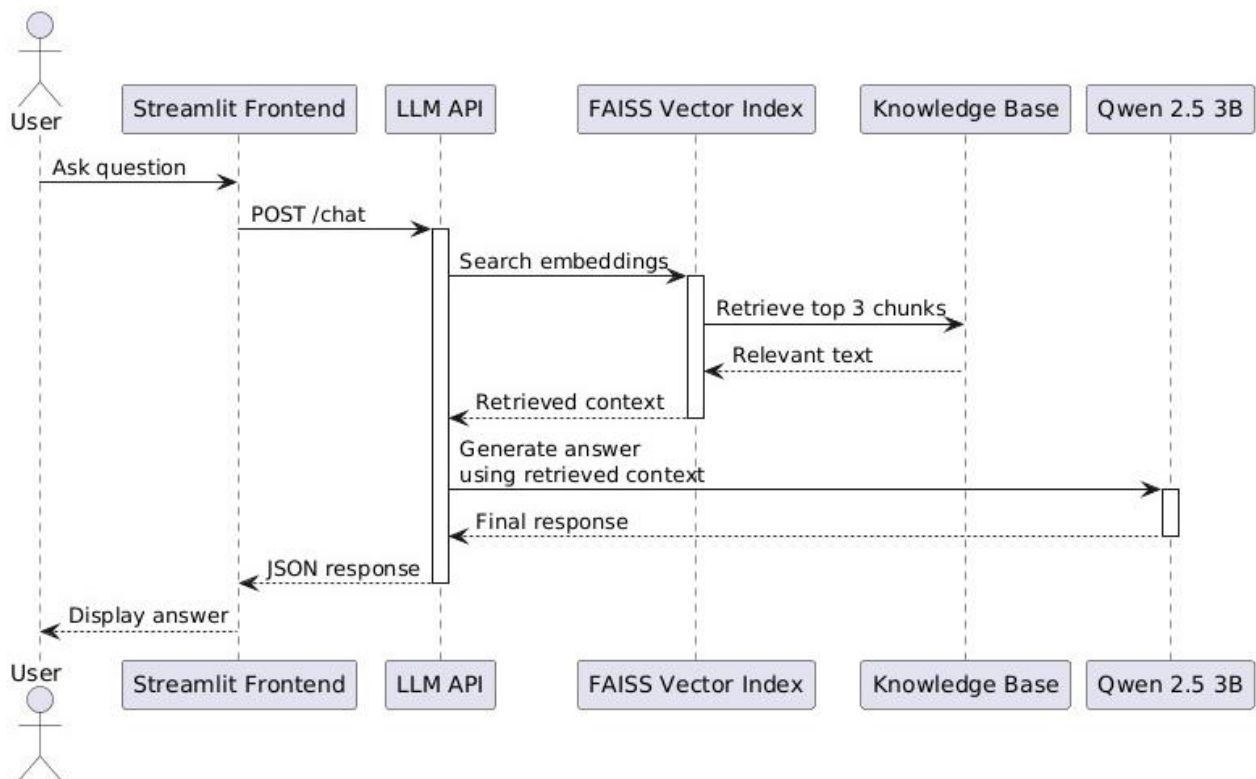
B. Drug Details



C. Price Prediction

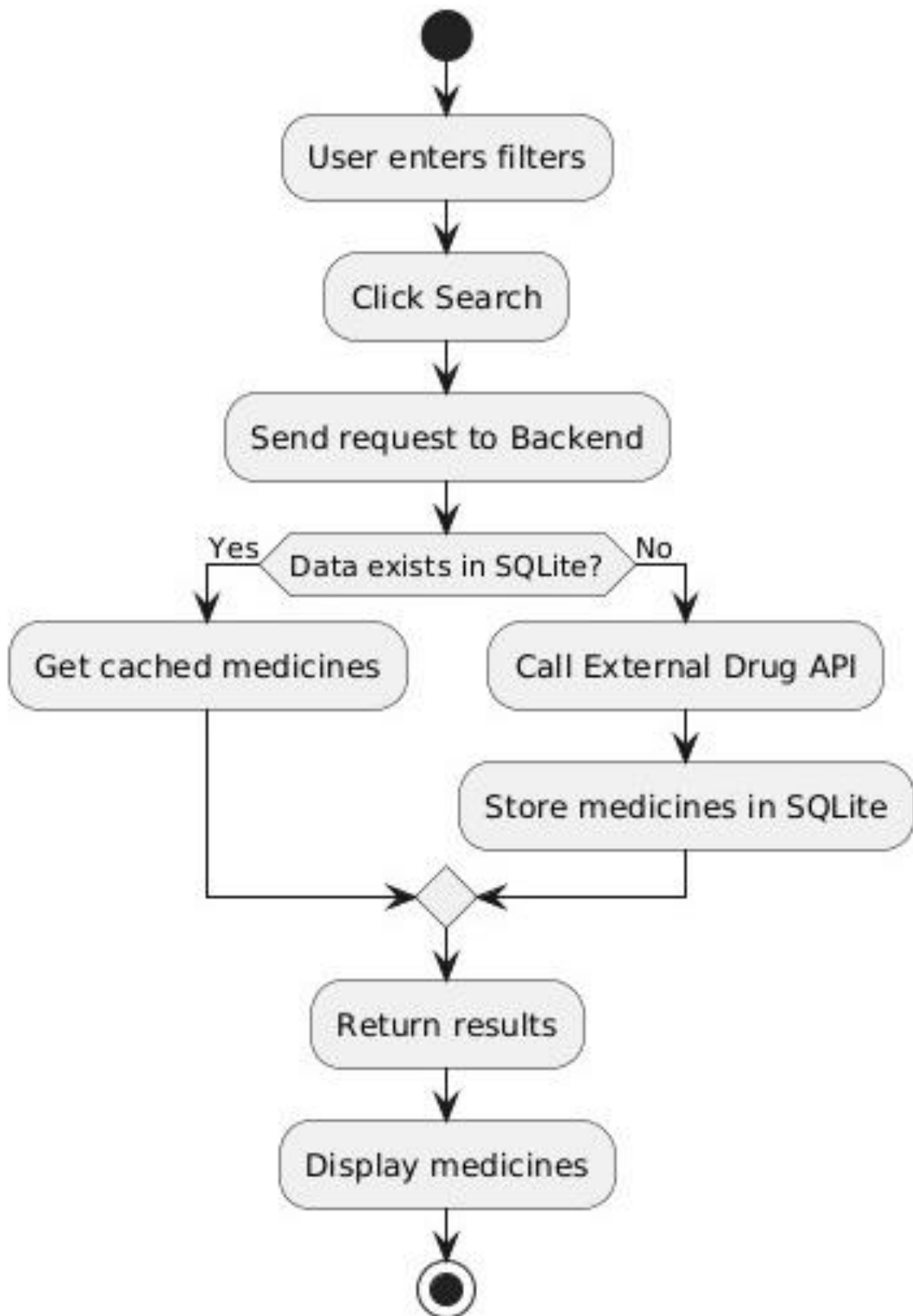


D. AI Chat (Qwen + RAG)

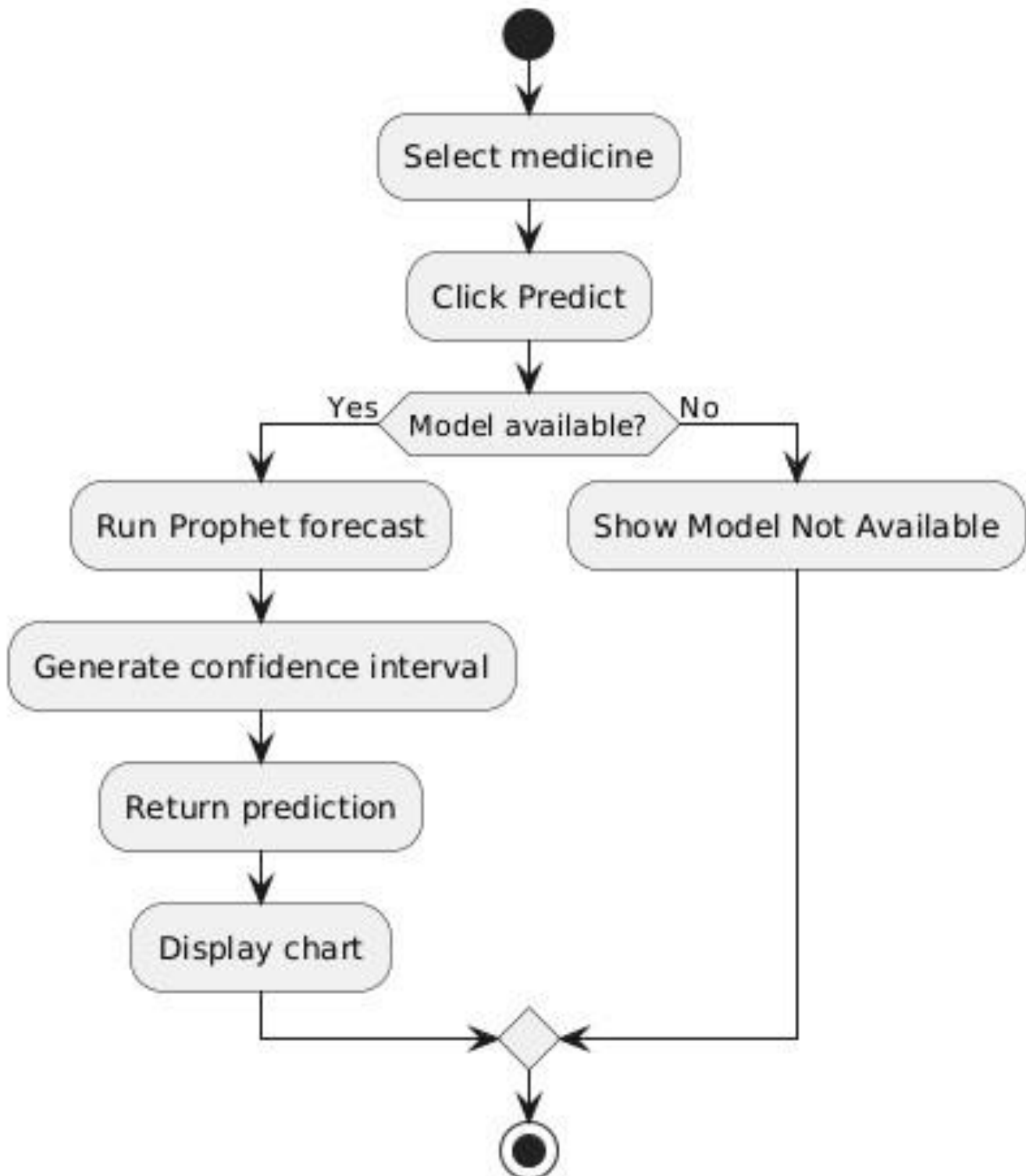


4.9 Activity Diagram

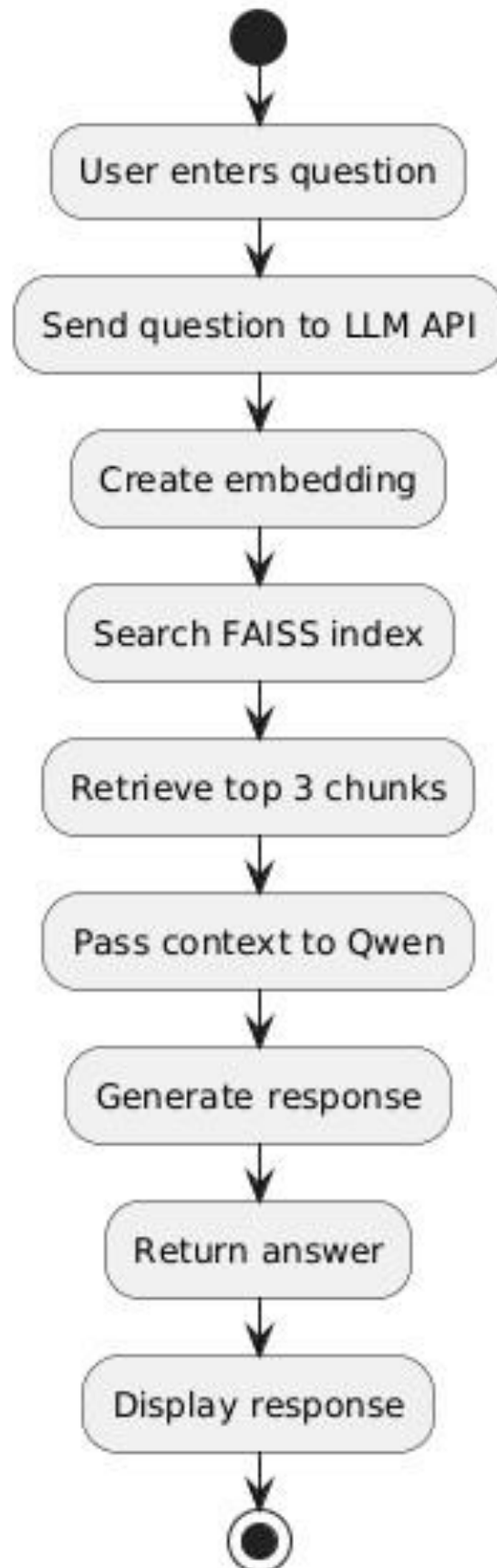
A. Drug Search



B. Price Prediction

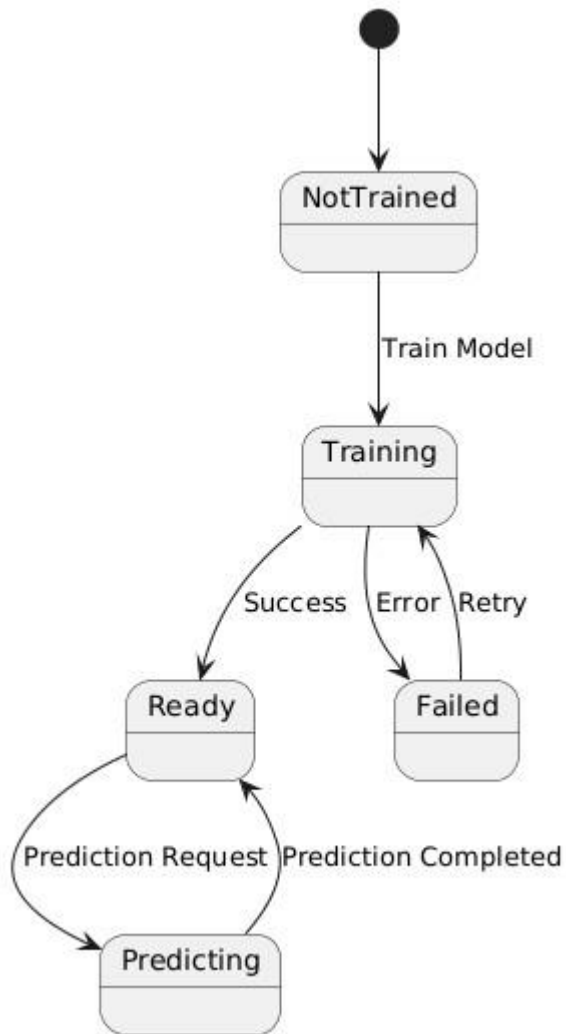


C. AI Assistant

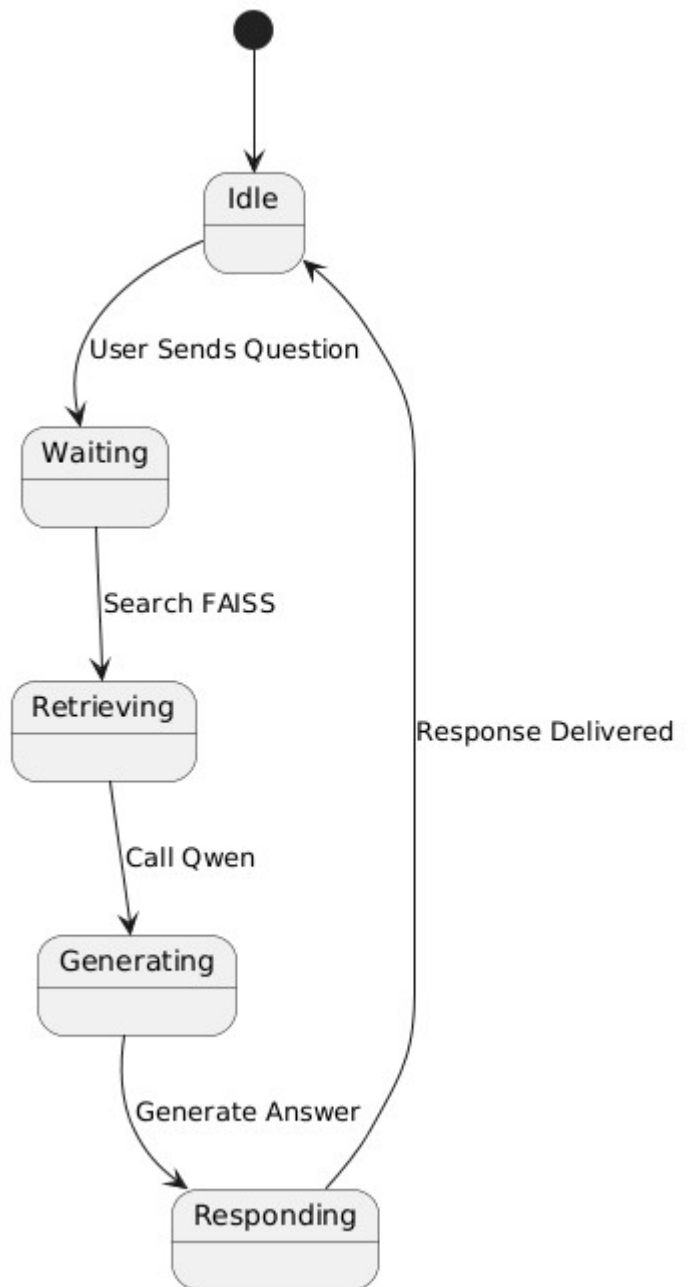


4.10 State Diagram

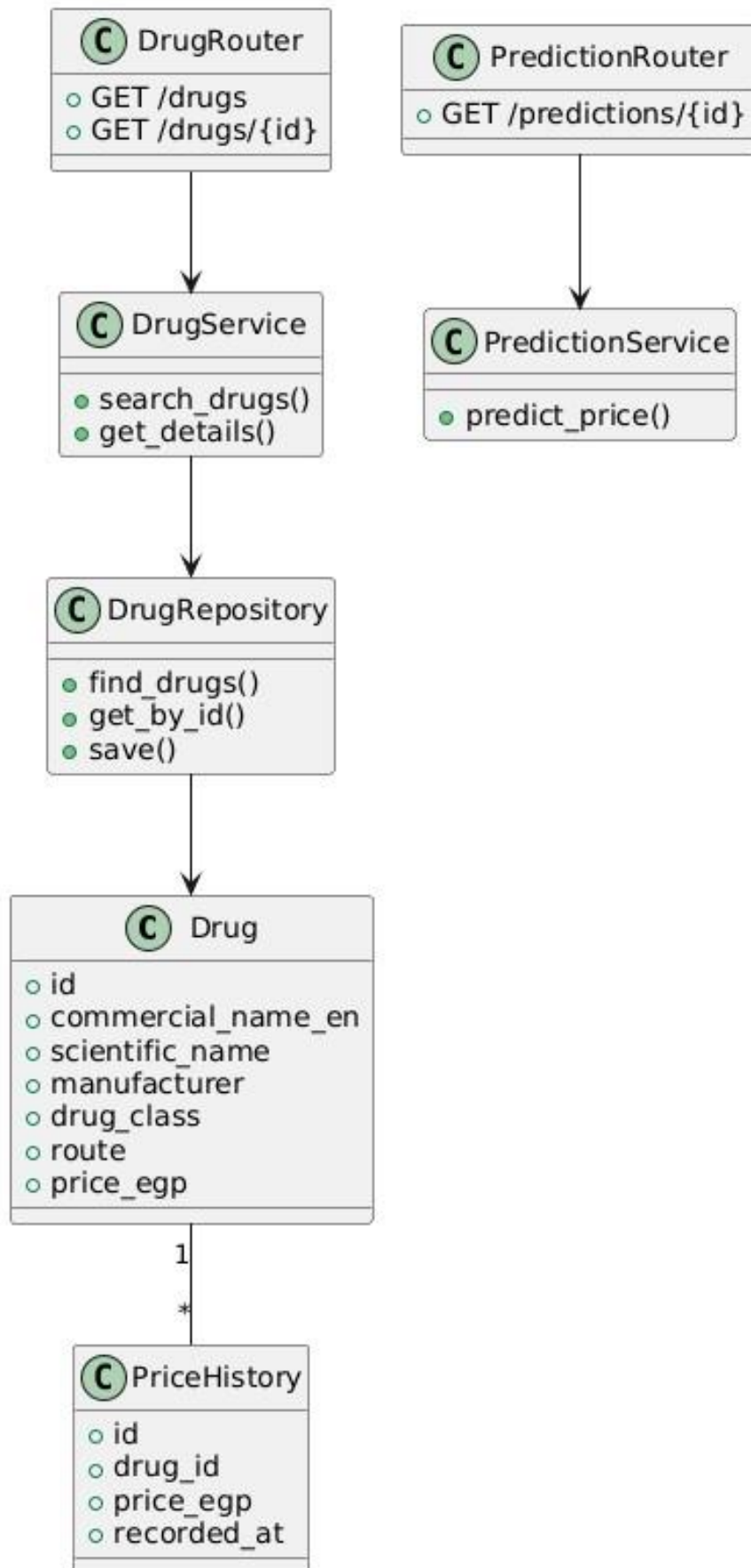
A. Drug Prediction Model



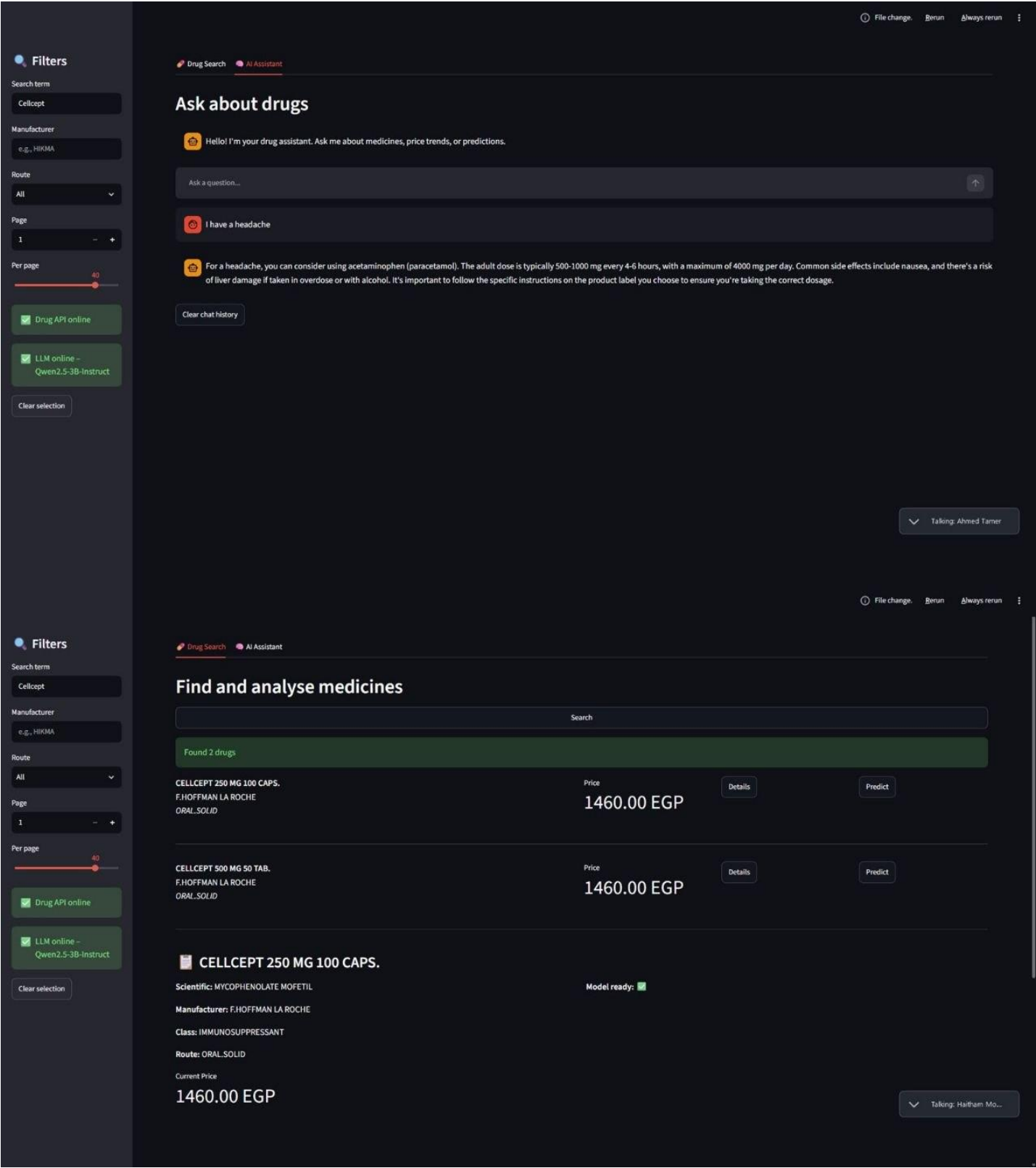
B. Chat Session



4.11 Class Diagram



4.12 UI/UX Design & Prototyping





UI/UX Guidelines

The user interface follows a clean and minimal design to simplify interaction for healthcare professionals and general users.

Design principles include:

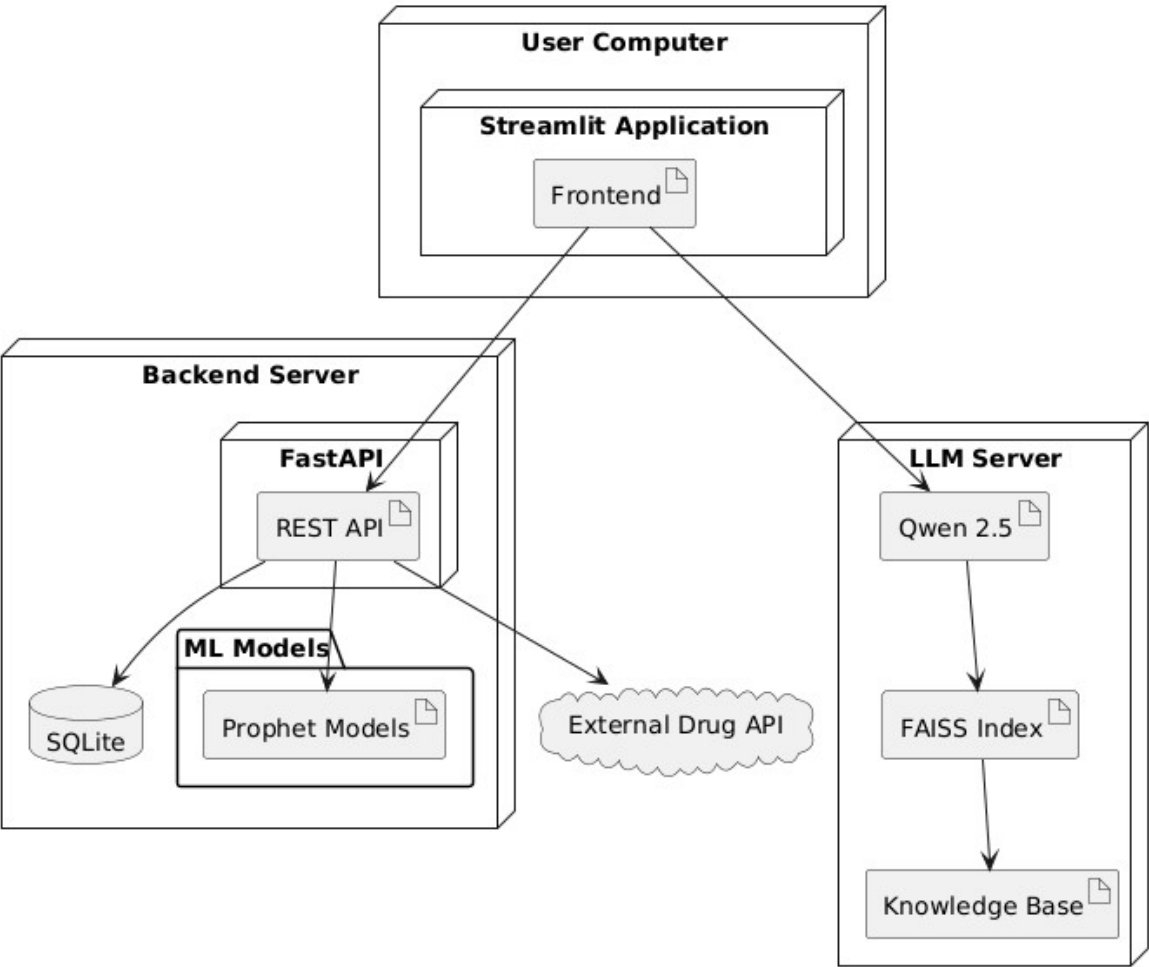
- Consistent layout across all pages.
- Clear navigation using Streamlit tabs and sidebar controls.
- Responsive design for different screen sizes.
- Readable typography with appropriate spacing.
- Interactive charts using Plotly for price visualization.
- Color coding to distinguish normal information, warnings, and errors.
- Accessibility considerations such as descriptive labels, sufficient color contrast, and feedback messages for loading and error states.

4.13 System Deployment & Integration

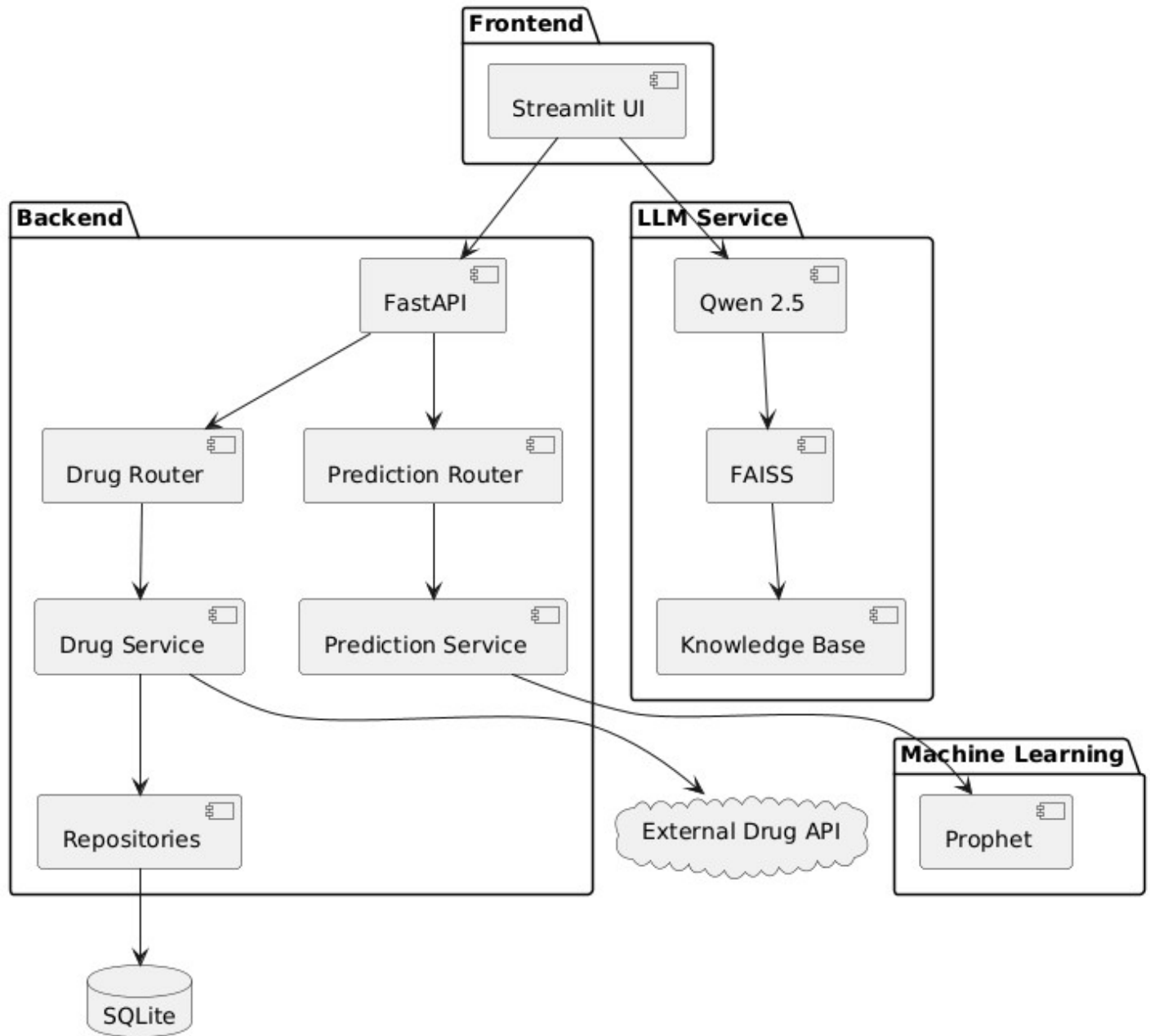
Technology Stack

Layer	Technology
Frontend	Streamlit
Backend	FastAPI
Database	SQLite
ORM	SQLAlchemy
Machine Learning	Prophet
AI Chatbot	Qwen 2.5 3B-Instruct
Embeddings	sentence-transformers/all-MiniLM-L6-v2
Vector Search	FAISS
Data Processing	Pandas, NumPy
Charts	Plotly
HTTP Client	httpx
API Testing	Swagger UI
Public AI Endpoint	Cloudflare Tunnel

Deployment Diagram



Component Diagram



4.14 Additional Deliverables

API Documentation

The backend exposes RESTful APIs documented automatically through FastAPI's OpenAPI/Swagger interface.

Main endpoints include:

Method	Endpoint	Description
GET	/drugs	Search and filter medicines
GET	/drugs/{id}	Retrieve detailed information for a medicine
GET	/predictions/{id}	Generate future price forecasts using Prophet
POST	/chat (<i>LLM service</i>)	Submit a user question to the Qwen RAG chatbot

Testing & Validation

The project should be validated using three testing levels:

- **Unit Testing:** Verify individual backend services, repositories, and prediction functions.
- **Integration Testing:** Ensure correct communication between the Streamlit frontend, FastAPI backend, SQLite database, Prophet model, and Qwen chatbot service.
- **User Acceptance Testing (UAT):** Confirm that end users can successfully search medicines, view details, generate forecasts, and interact with the AI assistant according to the project requirements.

Deployment Strategy

The application is designed for local deployment during development, with the possibility of cloud deployment in production.

The deployment process consists of:

1. Launch the FastAPI backend service.
2. Initialize the SQLite database.
3. Train or load Prophet forecasting models.
4. Start the Qwen RAG chatbot service and expose it through Cloudflare Tunnel if remote access is required.
5. Launch the Streamlit frontend and connect it to both the backend API and the chatbot endpoint.
6. Validate connectivity and API health before making the system available to users.

This modular deployment approach allows each subsystem (backend, forecasting, and chatbot) to be maintained and updated independently while remaining integrated through REST APIs.